

Question No:2

Question description

Udaya family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings.

Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input

The first input line has two integers n and m : the number of family members and toppings. The toppings are numbered $1, 2, \dots, m$.

After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x " (topping x is good) or "- x " (topping x is bad).

Output

Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution.

If there are no valid solutions, print "IMPOSSIBLE".

Constraints

- $1 \leq n, m \leq 10^5$
- $1 \leq x \leq m$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
```

EXPECTED OUTPUT

```
- + + + -
```

Test Case 2

INPUT (STDIN)

```
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
```

EXPECTED OUTPUT

```
- + - - +
```

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXN 100005
5  #define MAXE 200005
6
7  int n, m;
8
9  /* Graph */
10 int head[MAXN * 2];
11 int to[MAXE];
12 int nextEdge[MAXE];
13 int edgeCount = 0;
14
15 /* Reverse Graph */
16 int rhead[MAXN * 2];
17 int rto[MAXE];
18 int rnextEdge[MAXE];
19 int redgeCount = 0;
20
21 /* DFS */
22 int visited[MAXN * 2];
23 int order[MAXN * 2];
24 int orderCount = 0;
25
26 /* SCC */
27 int component[MAXN * 2];
28 int compCount = 0;
```

```
30  /* For second DFS */
31  int stack[MAXN * 2];
32
33  /* Add edge u -> v */
34  void addEdge(int u, int v)
35  {
36      to[edgeCount] = v;
37      nextEdge[edgeCount] = head[u];
38      head[u] = edgeCount++;
39
40      rto[redgeCount] = u;
41      rnextEdge[redgeCount] = rhead[v];
42      rhead[v] = redgeCount++;
43  }
44
45  /* First DFS */
46  void dfs1(int u)
47  {
48      visited[u] = 1;
49
50      for (int e = head[u]; e != -1; e = nextEdge[e])
51      {
52          int v = to[e];
53
54          if (!visited[v])
55              dfs1(v);
```

```

55         dfs1(v);
56     }
57
58     order[orderCount++] = u;
59 }
60
61 /* Second DFS on reverse graph */
62 void dfs2(int start, int comp)
63 {
64     int top = 0;
65
66     stack[top++] = start;
67     component[start] = comp;
68
69     while (top > 0)
70     {
71         int u = stack[--top];
72
73         for (int e = rhead[u]; e != -1; e = rnextEdge[e])
74         {
75             int v = rto[e];
76
77             if (component[v] == -1)
78             {
79                 component[v] = comp;
80                 stack[top++] = v;
81             }
82         }
83     }
84 }
85
86 /*
87     Variable x:
88
89     x = 1  -> topping included (+)
90     x = 0  -> topping not included (-)
91
92     Node representation:
93         2*i      -> variable i is FALSE
94         2*i + 1 -> variable i is TRUE
95 */
96
97 int getNode(int topping, char sign)
98 {
99     int x = topping - 1;
100
101     if (sign == '+')
102         return 2 * x + 1;

```

```

103         else
104             return 2 * x;
105     }
106
107     /*
108         Add clause:
109         (a OR b)
110
111         Equivalent implications:
112         NOT a -> b
113         NOT b -> a
114     */
115     void addClause(int a, int b)
116     {
117         int notA = a ^ 1;
118         int notB = b ^ 1;
119
120         addEdge(notA, b);
121         addEdge(notB, a);
122     }
123
124     int main()
125     {
126         scanf("%d %d", &n, &m);
127
128         int nodes = 2 * m;
129

```

```

130      /* Initialize graphs */
131      for (int i = 0; i < nodes; i++)
132      {
133          head[i] = -1;
134          rhead[i] = -1;
135          component[i] = -1;
136      }
137
138      /*
139       Read wishes.
140
141       Example:
142       + 1 + 2
143
144       means:
145       topping 1 OR topping 2
146      */
147
148      for (int i = 0; i < n; i++)
149      {
150          char sign1, sign2;
151          int topping1, topping2;
152
153          scanf(" %c %d %c %d",
154              &sign1, &topping1,
155              &sign2, &topping2);
156
157          int a = getNode(topping1, sign1);
158          int b = getNode(topping2, sign2);

```

```
159         addClause(a, b);
160     }
161
162     /* First DFS */
163     for (int i = 0; i < nodes; i++)
164     {
165         if (!visited[i])
166             dfs1(i);
167     }
168
169     /* Second DFS in reverse order */
170     for (int i = nodes - 1; i >= 0; i--)
171     {
172         int u = order[i];
173
174         if (component[u] == -1)
175         {
176             dfs2(u, compCount++);
177         }
178     }
179
180     /*
181     Check whether variable and its opposite
182     belong to the same SCC.
183     */
184     for (int i = 0; i < m; i++)
185     {
186         int falseNode = 2 * i;
```

```

188         int trueNode = 2 * i + 1;
189
190         if (component[falseNode] == component[trueNode])
191         {
192             printf("IMPOSSIBLE\n");
193             return 0;
194         }
195     }
196
197     /*
198     Construct answer.
199
200     If TRUE node has a higher SCC number than
201     FALSE node, choose TRUE.
202     */
203     for (int i = 0; i < m; i++)
204     {
205         int falseNode = 2 * i;
206         int trueNode = 2 * i + 1;
207
208         if (component[trueNode] > component[falseNode])
209             printf("+ ");
210         else
211             printf("- ");
212     }
213
214     printf("\n");
215
216     return 0;
217 }

```


Output:

Test Case-1:

```
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
- + + + -
```

Test Case-2:

```
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
- + - - +
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\DSA Practice>
```